# AURORA PAY — DOCUMENTAÇÃO TÉCNICA

---

## INFORMAÇÕES INSTITUCIONAIS

- **Instituição:** UNIPÊ — Centro Universitário de João Pessoa
- **Disciplina:** Análise de Projeto e Sistemas (APS)
- **Domínio:** Fintech
- **Data:** Novembro de 2026 | **Versão:** 2.0 (Documento Técnico Final Concluído)
- **Integrantes:** Rafael Rufino, Gabriel Cândido e Ighor Lucena
- **Slogan:** *"Seu dinheiro em movimento e sob controle."*

---

## CONTROLE DO DOCUMENTO E MÉTODO DE MODELAGEM

- **Objetivo:** Especificar a fintech Aurora Pay antes da implementação, da concepção ao handoff completo para o desenvolvedor (Dia 1 do Dev).
- **Fontes & Ferramentas:**
  - **UML e C4 Model:** Modelados no `draw.io` (app.diagrams.net).
  - **Banco de Dados:** PostgreSQL 16 gerido e validado via `pgAdmin`.
  - **Documentação & Handoff:** Estruturado e publicado no `Google Docs`.
  - **Custos Reais de Infraestrutura:** Calculados com a ferramenta oficial `calculator.aws` (região `sa-east-1`).
  - **Pesquisa Salarial:** Dados reais extraídos do `glassdoor.com.br` para a localidade.
- **Arquitetura:** Monólito Modular em Java 21 / Spring Boot 3, PostgreSQL 16, REST/JSON, React Native (Mobile), React/Vite (Web Admin) e AWS (sa-east-1) em ECS Fargate.

---

# BLOCO 1 — CONCEPÇÃO E ESCOPO DO PRODUTO

### 1.1 Documento de Visão

- **Nome / Slogan:** AURORA PAY — Seu dinheiro em movimento e sob controle.

- **Domínio:** Fintech (carteira digital, pagamentos PIX/boleto, controle financeiro, crédito assistido e conciliação para lojistas).
- **Problema Real:** Autônomos e freelancers enfrentam volatilidade na gestão de renda e conciliação financeira; pequenos comerciantes sofrem com perda de tempo e erros em conciliações manuais do PIX; usuários estão expostos a fraudes de tomada de conta e falta de transparência em tarifas.
- **Proposta de Valor:** Carteira conectada a BaaS com conta transacional, PIX com liquidação em tempo real, boletos bancários, cartão virtual tokenizado, conciliação em lote automatizada para lojistas e ledger imutável de dupla entrada.
- **Fora de Escopo (MVP):** Emissão de cartões físicos com chip, investimentos em renda variável/criptomoedas, processamento próprio de arranjos de pagamento sem BaaS parceiro.

## 1.2 Personas Detalhadas (Mínimo 3 Personas Exigidas)

1. **Persona 1 — João Silva (Autônomo / Freelancer, 29 anos):**
    - *Contexto:* Designer gráfico freelancer com renda mensal flutuante (R$ 3.000 a R$ 8.000).
    - *Necessidade:* Precisa segurar dinheiro para despesas fixas e impostos separadamente da conta corrente principal, automatizar cobranças a clientes via PIX e acompanhar o fluxo de caixa sem planilhas complexas.
2. **Persona 2 — Maria Oliveira (Pequena Comerciante, 42 anos):**
    - *Contexto:* Proprietária de uma loja de roupas artesanais com vendas presenciais e online.
    - *Necessidade:* Recebe dezenas de pagamentos PIX por dia e perde horas no fim do dia cruzando extratos bancários com comprovantes enviados por clientes via mensagem. Precisa de conciliação automática idempotente via QR Code dinâmico.
3. **Persona 3 — Carlos Eduardo (Analista de Compliance e Antifraude, 35 anos):**
    - *Contexto:* Profissional interno de operações de risco na fintech.
    - *Necessidade:* Precisa de um painel web administrativo seguro (RBAC) para visualizar alertas em tempo real de transações suspeitas, analisar decisões automatizadas do engine de risco e liberar/bloquear contas retenção de segurança.

## 1.3 Mapa de Stakeholders

- **Clientes Final (PF):** Usuários da carteira digital; exigem alta disponibilidade, usabilidade simples e segurança.
- **Lojistas / Merchants (PJ/MEI):** Usuários comerciais; exigem conciliação imediata e relatórios transparentes.
- **Mesa de Compliance e Risco:** Operadores internos; exigem auditoria imutável, rastreabilidade e alertas preditivos.

- **Parceiro BaaS (Banking as a Service):** Instituição financeira licenciada pelo BCB; provê infraestrutura bancária, DICT e emissão de moeda eletrônica.
- **Órgãos Reguladores (BCB / COAF / ANPD):** Exigem cumprimento rigoroso de PLD/FT, relatórios de auditoria e adequação total à LGPD.

## 1.4 Estudo de Viabilidade e Parecer Jurídico BaaS / LGPD

- **Viabilidade Técnica e Operacional:** O uso de monólito modular reduz sobrecarga de microsserviços. A hospedagem serverless em AWS ECS Fargate elimina gestão de clusters EC2.
- **Papel Regulatório BaaS:** O Aurora Pay opera como Interface de Tecnologia e Pagamentos (Correspondente Bancário). A custódia de moeda eletrônica, chave PIX no DICT e liquidação pertencem à Instituição Parceira habilitada pelo Banco Central.
- **Adequação LGPD:**
    - *Bases Legais:* Execução de contrato (Art. 7º, V) para pagamentos e Cumprimento de Obrigação Regulatória (Art. 7º, II) para retenção de registros e KYC.
    - *Tratamento PII:* Dados sensíveis (CPF, e-mail, telefone) usam hashes HMAC (SHA-256) e criptografia AES-256-GCM via AWS KMS. O sistema proíbe armazenamento de senhas abertas ou dados brutos de cartões.

## 1.5 Matriz de Riscos Iniciais (5 Riscos Fundamentais)

| **Risco** | **Categoria** | **Probabilidade** | **Impacto** | **Plano de Mitigação** |
|---|---|---|---|---|
| R01: Indisponibilidade na API do Provedor BaaS | Técnico / Externo | Média | Alto | Implementação de Circuit Breaker (Resilience4j), retentativas com backoff exponencial e fila de contingência (SQS FIFO). |
| R02: Fraude de Onboarding com Documentos Falsos | Segurança / Operacional | Média | Alto | Validação automatizada em 2 camadas via provedor KYC (Liveness Test + OCR) |

| Risco | Categoria | Probabilidade | Impacto | Plano de Mitigação |
|---|---|---|---|---|
| | | | | com transição para analista em `MANUAL_REVIEW`. |
| R03: Inconsistência de Saldo por Concorrência | Técnico / Financeiro | Baixa | Crítico | Motor contábil de Dupla Entrada (*Double-Entry Ledger*) com isolamento de transações em banco relacional e locks pessimistas. |
| R04: Extrapolação do Orçamento AWS na Fase Inicial | Financeiro | Média | Médio | Definição de AWS Budgets com alertas de consumo em 80%, auto-scaling em ECS Fargate baseado em CPU/memória e desligamento de Dev/Staging à noite. |
| R05: Não conformidade com solicitações da LGPD | Regulatório | Baixa | Alto | Implementação de módulo audit/iam específico para atender revogações de consentimento e exportação de dados em até 15 dias úteis. |

# BLOCO 2 — REQUISITOS DO SISTEMA

## 2.1 Requisitos Funcionais com Priorização MoSCoW (28 RFs)

- **RF001 (Must):** Cadastrar cliente PF (Nome, CPF, E-mail, Telefone, aceite de Termos). *Justificativa:* Essencial para a existência da conta.
- **RF002 (Must):** Submeter dados ao provedor de KYC e armazenar score de validação. *Justificativa:* Obrigatoriedade regulatória anti-fraude.
- **RF003 (Must):** Autenticar usuário via Keycloak (OIDC JWT com PKCE, expiração de 15 min). *Justificativa:* Segurança básica de acesso.
- **RF004 (Must):** Exigir reautenticação/MFA em operações transacionais de risco ou alto valor.
- **RF005 (Must):** Consultar saldo disponível, saldo reservado e status da conta.
- **RF006 (Must):** Exibir extrato paginado e filtrável por período e tipo de transação.
- **RF007 (Must):** Cadastrar, consultar e remover chaves PIX via integração BaaS.
- **RF008 (Must):** Iniciar transferência PIX exigindo obrigatoriamente o cabeçalho HTTP `Idempotency-Key`. *Justificativa:* Evita duplicação de pagamentos.
- **RF009 (Must):** Processar e atualizar ciclo de vida do PIX (CREATED, PENDING, IN_DOUBT, SETTLED, REJECTED).
- **RF010 (Must):** Gerar cobrança PIX com QR Code dinâmico e TXID único.
- **RF011 (Should):** Consultar e pagar boletos bancários com validação do beneficiário.
- **RF012 (Should):** Emitir cartão virtual tokenizado e alternar estado (Ativo/Bloqueado).
- **RF013 (Should):** Categorizar automaticamente lançamentos financeiros.
- **RF014 (Should):** Exibir resumo financeiro mensal consolidado.
- **RF015 (Could):** Criar e gerenciar metas financeiras/cofrinhos com segregação lógica de saldo.
- **RF016 (Should):** Simular crédito pessoal detalhando Custo Efetivo Total (CET), taxa mensal e parcelas.
- **RF017 (Should):** Submeter proposta de crédito para motor de decisão automatizado.
- **RF018 (Should):** Registrar versões da política de crédito e trilha de aprovação/recusa.
- **RF019 (Must):** Cadastrar perfil de Lojista/Merchant vinculado à conta.
- **RF020 (Must):** Criar cobranças de vendas para lojistas com TXID e data de expiração.
- **RF021 (Must):** Conciliar webhooks de pagamento recebidos do BaaS de forma idempotente.
- **RF022 (Must):** Calcular score de risco transacional em milissegundos antes do envio da ordem.
- **RF023 (Must):** Aplicar ações de risco automatizadas (APPROVE, CHALLENGE, REVIEW, BLOCK).
- **RF024 (Must):** Criar e gerenciar alertas de suspeita de fraude na mesa de compliance.
- **RF025 (Must):** Auditar requisições financeiras com correlation ID e trace ID distribuído.

- **RF026 (Must):** Gerenciar papéis e permissões de acesso administrativo (RBAC).
- **RF027 (Should):** Permitir gestão de consentimentos pelo titular dos dados.
- **RF028 (Must):** Atender e registrar solicitações de titulares no âmbito da LGPD.

## 2.2 Requisitos Não Funcionais (22 RNFs com Métricas Numéricas)

- **RNF001:** Latência de leitura de API p95 <= 300 ms sob carga contínua de 150 TPS.
- **RNF002:** Tempo de processamento interno do PIX p95 <= 4 s até o envio ao parceiro BaaS.
- **RNF003:** Resposta do Motor de Risco local p95 <= 150 ms por avaliação.
- **RNF004:** Suportar 10.000 usuários ativos mensais e perfil sustentado de 150 TPS.
- **RNF005:** Auto-scaling capaz de triplicar a capacidade para 450 TPS em <= 5 minutos.
- **RNF006:** Disponibilidade dos serviços (SLA/Uptime) de 99,9% ao mês (downtime máx. 43 min/mês).
- **RNF007:** Metas de Recuperação de Desastres: RTO <= 30 min e RPO <= 5 min.
- **RNF008:** Criptografia TLS 1.3 em trânsito e AES-256-GCM em repouso via AWS KMS.
- **RNF009:** Tokens de acesso com validade de 15 min; hashes de senha com Argon2id.
- **RNF010:** Rate limiting por IP/Usuário de máximo 100 requisições/minuto nas rotas públicas.
- **RNF011:** Trilha de auditoria *append-only* imutável e à prova de adulteração.
- **RNF012:** Proibição de armazenamento de dados brutos de cartão (PAN/CVV) ou senhas em texto puro.
- **RNF013:** Atendimento a requisições de titulares da LGPD efetuado em prazo <= 15 dias úteis.
- **RNF014:** Rotação automatizada de segredos no AWS Secrets Manager a cada 90 dias.
- **RNF015:** Expurgo/anonimização de logs após 5 anos (retenção regulatória).
- **RNF016:** Garantia do Princípio da Contabilidade Dupla: $\sum \text{Débitos} + \sum \text{Créditos} = 0$ por entrada no Ledger.
- **RNF017:** Compatibilidade garantida nos sistemas Android 9+, iOS 15+ e navegadores web modernos.
- **RNF018:** Onboarding completo finalizado pelo usuário em tempo <= 5 minutos.
- **RNF019:** Cobertura de Tracing Distribuído (OpenTelemetry) em 100% das requisições com `trace_id`.
- **RNF020:** Cobertura de testes unitários e de integração >= 80% nos módulos core do domínio.
- **RNF021:** Entrega de eventos de domínio via padrão *Transactional Outbox* com garantia *at-least-once*.
- **RNF022:** Interface acessível em conformidade com as diretrizes WCAG 2.2 Nível AA.

## 2.3 Especificação das Histórias de Usuário (15 USs Mapeadas com BDD Given/When/Then e Story Points)

1. **US01 — Cadastro de Cliente PF (8 Story Points):**

- *História:* Como João Silva, quero cadastrar meus dados pessoais no aplicativo, para abrir uma conta digital no Aurora Pay.
- *Critérios de Aceite:*
  - **Dado que** estou na tela de onboarding e informo Nome, CPF válido, E-mail e Telefone; **Quando** submeto o formulário e aceito os termos; **Então** o sistema cria o cadastro com status `PENDING_KYC` e dispara a validação de identidade.

2. **US02 — Autenticação Segura MFA (5 Story Points):**
   - *História:* Como cliente cadastrado, quero me autenticar usando biometria ou senha com MFA, para acessar minha conta com segurança.
   - *Critérios de Aceite:*
     - **Dado que** informo minhas credenciais válidas; **Quando** o sistema solicita a segunda etapa de validação (MFA); **Então** recebo o token OIDC JWT (15 min) e acesso o aplicativo.
3. **US03 — Consulta de Saldo e Extrato Paginado (3 Story Points):**
   - *História:* Como João Silva, quero consultar meu saldo e extrato detalhado, para acompanhar minhas movimentações diárias.
   - *Critérios de Aceite:*
     - **Dado que** estou autenticado; **Quando** solicito a tela de extrato com filtro dos últimos 30 dias; **Então** o saldo disponível é exibido instantaneamente (<300ms) com a lista de lançamentos paginada.
4. **US04 — Cadastro e Gestão de Chaves PIX (5 Story Points):**
   - *História:* Como cliente, quero cadastrar minha chave PIX (CPF, E-mail ou Aleatória), para receber transferências rapidamente.
   - *Critérios de Aceite:*
     - **Dado que** solicito a criação de uma nova chave PIX; **Quando** confirmo o tipo de chave e autentico com MFA; **Então** o sistema registra a chave junto ao BaaS parceiro e confirma o vínculo na minha conta.
5. **US05 — Transferência PIX com Idempotência (13 Story Points):**
   - *História:* Como João Silva, quero transferir um valor por PIX com garantia de idempotência, para não pagar duas vezes caso a conexão oscile.
   - *Critérios de Aceite:*
     - **Dado que** informo a chave de destino, o valor e o cabeçalho `Idempotency-Key`; **Quando** confirmo a transferência; **Então** o Ledger efetua o débito e o valor é enviado ao BaaS parceiro sem risco de duplicação.
6. **US06 — Geração de QR Code PIX Dinâmico para Lojista (8 Story Points):**
   - *História:* Como Maria Oliveira, quero gerar um QR Code dinâmico com valor específico e TXID, para receber pagamentos de clientes na minha loja.
   - *Critérios de Aceite:*
     - **Dado que** acesso a área do merchant e digito o valor de venda R$ 150,00; **Quando** clico em gerar cobrança; **Então** o sistema gera o QR Code PIX contendo TXID único e status `CREATED`.

7. **US07 — Conciliação Automática de Webhook PIX (8 Story Points):**
    - *História:* Como Maria Oliveira, quero que as vendas PIX recebidas sejam conciliadas automaticamente, para não precisar conferir comprovantes manuais.
    - *Critérios de Aceite:*
        - **Dado que** o BaaS notifica o recebimento de um PIX via Webhook; **Quando** o Aurora Pay recebe o payload contendo o TXID; **Então** a venda é marcada como `SETTLED` e o saldo creditado na conta da lojista via Ledger.
8. **US08 — Simulação e Aprovação de Crédito Pessoal (8 Story Points):**
    - *História:* Como cliente, quero simular um empréstimo pessoal com exibição clara do CET, para solicitar crédito de forma transparente.
    - *Critérios de Aceite:*
        - **Dado que** informo o valor desejado (ex: R$ 5.000) e prazo de 12 meses; **Quando** solicito a simulação; **Então** o sistema exibe o valor da parcela, taxa de juros e Custo Efetivo Total (CET).
9. **US09 — Emissão de Cartão Virtual Tokenizado (5 Story Points):**
    - *História:* Como cliente, quero gerar um cartão virtual temporário, para realizar compras online com segurança.
    - *Critérios de Aceite:*
        - **Dado que** solicito a criação de um cartão virtual; **Quando** me autentico com MFA; **Então** o sistema gera o token do cartão com data de expiração e CVV dinâmico sem salvar dados sensíveis em texto puro.
10. **US10 — Bloqueio Temporário de Cartão Virtual (2 Story Points):**
    - *História:* Como cliente, quero bloquear temporariamente meu cartão virtual pelo aplicativo, para evitar usos não autorizados.
    - *Critérios de Aceite:*
        - **Dado que** possuo um cartão ativo; **Quando** aciono a opção "Bloquear Cartão"; **Então** o status do cartão passa para `BLOCKED` e autorizações de compra são rejeitadas.
11. **US11 — Avaliação Antifraude e Score de Risco (13 Story Points):**
    - *História:* Como sistema, quero avaliar o risco de cada transação PIX antes da execução, para prevenir fraudes transacionais.
    - *Critérios de Aceite:*
        - **Dado que** uma requisição de transferência PIX é iniciada; **Quando** o motor de risco avalia o valor, dispositivo e histórico (<150ms); **Então** retorna a decisão `APPROVE`, `REVIEW` ou `BLOCK`.
12. **US12 — Painel da Mesa de Compliance e Alertas de Fraude (8 Story Points):**
    - *História:* Como Carlos Eduardo, quero visualizar alertas de suspeita de fraude em um painel web administrativo, para analisar transações retidas.
    - *Critérios de Aceite:*
        - **Dado que** uma transação recebe a decisão `REVIEW`; **Quando** acesso a fila de compliance no Web Admin; **Então** posso visualizar o score, o motivo do alerta e aprovar ou rejeitar a operação.

13. **US13 — Gestão de Permissões Administrativas RBAC (5 Story Points):**
    - *História:* Como administrador do sistema, quero conceder papéis de acesso granulares, para garantir que analistas vejam apenas dados autorizados.
    - *Critérios de Aceite:*
        - **Dado que** cadastro um novo colaborador interno; **Quando** atribuo o papel `ROLE_COMPLIANCE`; **Então** ele tem acesso restrito às telas de auditoria sem permissão de alterar parâmetros de sistema.
14. **US14 — Atendimento a Solicitação de Titular da LGPD (5 Story Points):**
    - *História:* Como João Silva, quero solicitar o relatório de dados pessoais retidos pelo sistema, para exercer meus direitos previstos na LGPD.
    - *Critérios de Aceite:*
        - **Dado que** solicito a exportação dos meus dados na área de privacidade; **Quando** a requisição é confirmada; **Então** o módulo de audit/iam gera um relatório estruturado em até 15 dias úteis.
15. **US15 — Extrato da Trilha de Auditoria com Correlation ID (5 Story Points):**
    - *História:* Como Carlos Eduardo, quero buscar a trilha de auditoria de uma transação específica via Correlation ID, para rastrear eventos em investigações.
    - *Critérios de Aceite:*
        - **Dado que** possuo o `correlation_id` de uma requisição; **Quando** pesquiso no módulo de auditoria; **Então** todos os logs imutáveis e eventos associados são exibidos em ordem cronológica.

---

# BLOCO 3 — ARQUITETURA DE SOFTWARE E MODELO C4

## 3.1 Decision Records da Arquitetura (ADRs)

- **ADR-001: Monólito Modular com Spring Boot 3 / Java 21**
    - *Status:* Aceito.
    - *Contexto:* Evitar complexidade operacional e latência de rede distribuída no MVP.
    - *Decisão:* Adotar Monólito Modular isolando módulos de domínio (`iam`, `ledger`, `payments`, `risk`, `credit`, `merchant`, `audit`). As fronteiras são validadas em tempo de compilação via Spring Modulith e ArchUnit.
    - *Alternativa Descartada:* Microsserviços Distribuídos (Descartado por introduzir complexidade desnecessária e transações distribuídas via Saga em estágio inicial).
- **ADR-002: Motor Contábil baseado em Ledger de Dupla Entrada (*Double-Entry Ledger*)**

    - *Status:* Aceito.
    - *Contexto:* Necessidade de garantir consistência contábil matemática absoluta.
    - *Decisão:* Cada transação deve criar um `JournalEntry` imutável contendo no mínimo 2 `Postings` (Débito e Crédito) em que a soma dos valores seja estritamente igual a zero.
    - *Alternativa Descartada:* Atualização direta de coluna `balance` em tabela de contas (Descartado por inconsistência sob alta concorrência e falta de auditabilidade contábil).
- **ADR-003 (Revisada): Migração de EKS para AWS ECS Fargate Serverless**
    - *Status:* Alterado / Aprovado.
    - *Contexto:* O Amazon EKS exigia custo fixo de US$ 73/mês apenas pela taxa de gerenciamento de cluster, além de alta complexidade de manutenção de nós EC2 e Kubernetes YAMLs.
    - *Decisão:* Adotar AWS ECS Fargate. O provisionamento totalmente serverless elimina a gestão de instâncias, reduz custos operacionais e simplifica auto-scaling via CloudWatch Alarms.
    - *Alternativa Descartada:* Amazon EKS (Descartado por custo e sobrecarga operacional desproporcional para a equipe).

## 3.2 Tabela da Stack Tecnológica Justificada

| **Camada** | **Tecnologia Escolhida** | **Alternativa Descartada** | **Motivo Técnico da Escolha** |
|---|---|---|---|
| **Backend Core** | Java 21 + Spring Boot 3 | Node.js (TypeScript) | Tipagem forte, ecossistema maduro para transações financeiras e suporte ao Spring Modulith. |
| **Mobile App** | React Native | Flutter | Compartilhamento de código e componentes de interface com o painel web em React/Vite. |
| **Web Admin** | React + Vite + Tailwind | Angular | Inicialização ultra-rápida, menor tamanho de bundle e alta produtividade de desenvolvimento. |

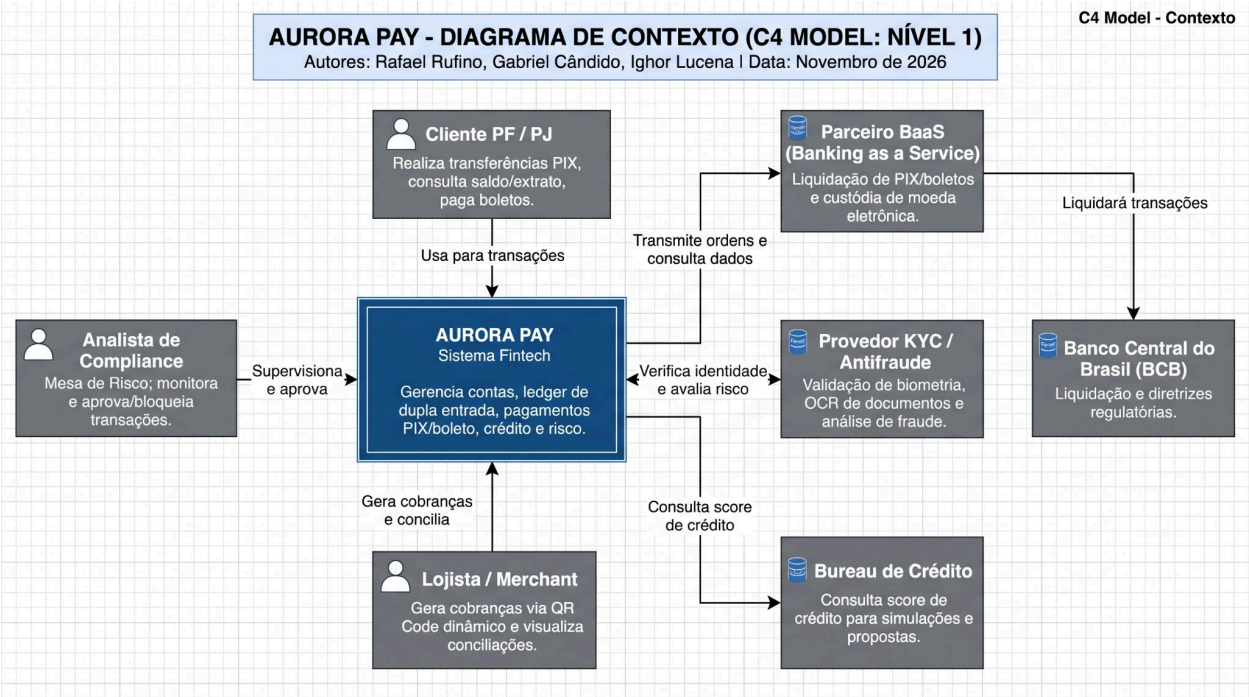
| Camada | Tecnologia Escolhida | Alternativa Descartada | Motivo Técnico da Escolha |
|---|---|---|---|
| **Banco Relacional** | PostgreSQL 16 | MongoDB | Garantia rigorosa de propriedades ACID, suporte nativo a schemas e extensões de criptografia (`pgcrypto`). |
| **Cache & Rate Limit** | Redis 7 | Memcached | Suporte a estruturas de dados complexas, controle de concorrência com Scripting Lua e persistência. |
| **IAM / Auth** | Keycloak (OIDC/OAuth2) | Auth0 / Cognito | Solução open-source sem custo por usuário (MAU), rodando em container próprio com suporte a PKCE. |
| **Hospedagem Cloud** | AWS ECS Fargate | GCP Cloud Run / EC2 | Ambiente altamente integrado, conformidade regulatória e facilidade de sizing serverless em sa-east-1. |

## 3.3 Especificação Completa do Modelo C4 (4 Níveis Hierárquicos Exigidos)

### C4 — Nível 1: Diagrama de Contexto (System Context)

- **Escopo:** Apresenta os usuários e sistemas externos interagindo com a fintech Aurora Pay.
- **Atores & Sistemas:**
  - `[Cliente PF]`: Usa o Mobile App para transferências, extratos e pagamentos.
  - `[Lojista / Merchant]`: Usa a interface Web/Mobile para cobranças e conciliação PIX.
  - `[Analista de Compliance]`: Usa o Web Admin para monitoramento de risco e auditoria.

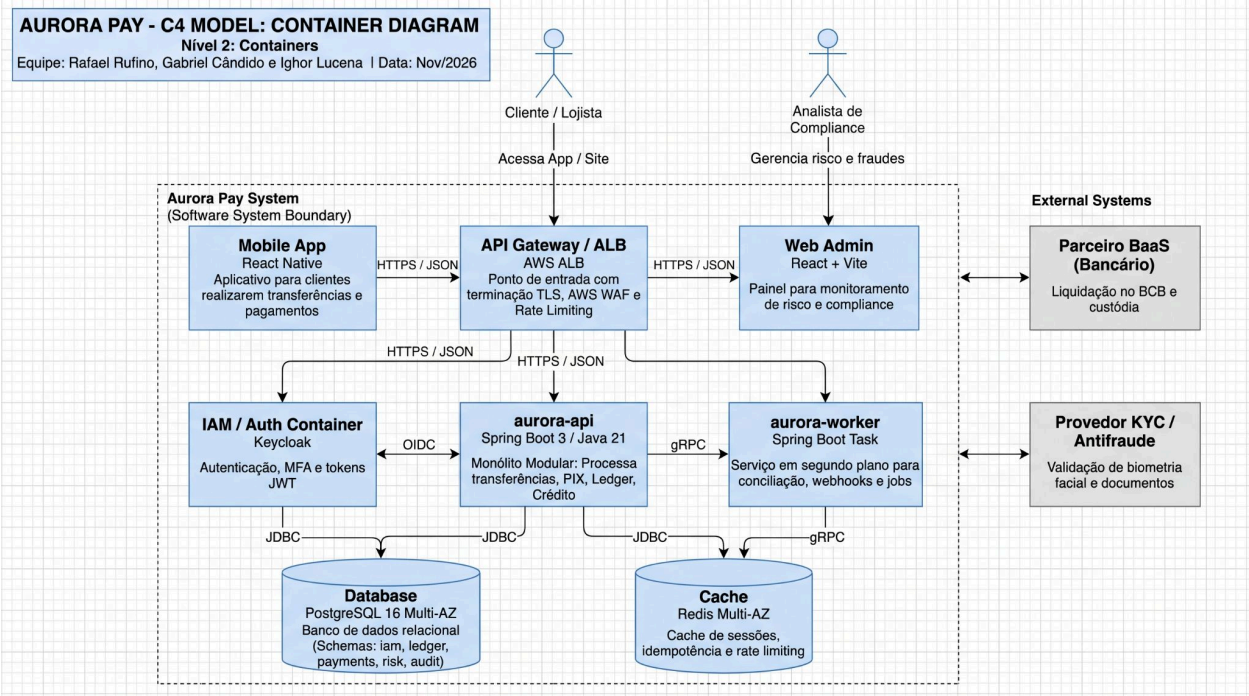
- `[Aurora Pay System]`: Sistema central fintech que gerencia contas, ledger e risco.
- `[Parceiro BaaS (Bancário)]`: Executa a liquidação das transações no BCB e guarda a moeda eletrônica.
- `[Provedor KYC / Antifraude]`: Valida biometria facial e OCR de documentos de identidade.
- `[Bureau de Crédito]`: Consulta score de crédito para empréstimos.




## C4 — Nível 2: Diagrama de Containers (Containers)

- **Escopo:** Detalha as aplicações e datastores que compõem o sistema Aurora Pay.
- **Containers:**
  - `Mobile App (React Native)`: Aplicativo móvel para clientes PF/PJ. Comunica-se via HTTPS/JSON REST.
  - `Web Admin (React/Vite)`: Painel administrativo para a equipe interna de compliance e operações.
  - `API Gateway / ALB (AWS ALB)`: Ponto de entrada com Terminação TLS 1.3 e AWS WAF.
  - `Keycloak Container (OIDC)`: Serviço de autenticação e emissão de tokens JWT.
  - `aurora-api (ECS Fargate Task)`: Monólito modular Spring Boot executando as regras de negócio do core financeiro.
  - `aurora-worker (ECS Fargate Task)`: Serviço em segundo plano para conciliação assíncrona, webhooks e jobs retidos.
  - `Database (PostgreSQL 16 Multi-AZ)`: Banco de dados relacional contendo os schemas segregados do sistema.

- `Cache (Redis Multi-AZ)`: Cache de sessões, tabela de idempotência e rate limiting.




## C4 — Nível 3: Diagrama de Componentes (Components - Módulo Core Ledger & Payments)

- **Escopo:** Decomposição interna do container `aurora-api` em módulos lógicos.
- **Componentes Mapeados:**
  - `PaymentController`: Expõe os endpoints REST para envio de PIX e geração de QR Code.
  - `IdempotencyFilter`: Valida e reserva a `Idempotency-Key` no Redis antes do processamento.
  - `RiskAssessmentAdapter`: Consulta o engine de risco local antes de aprovar a transação.
  - `LedgerPostingService`: Executa a reserva contábil em nível de banco de dados (`BEGIN`, `LOCK`, débitos/créditos).
  - `BaaSIntegrationAdapter`: Transmite a ordem de pagamento aprovada para o parceiro BaaS.
  - `OutboxPublisher`: Salva eventos de domínio na tabela `outbox_events` para publicação assíncrona.

C4 — Nível 4: Diagrama de Código (Code Snippet — Componente Crítico `LedgerPostingEngine`)

- **Escopo:** Implementação do motor contábil imutável garantindo a regra da contabilidade dupla ($\sum \text{Débitos} + \sum \text{Créditos} = 0$).

```java
package com.aurorapay.ledger.domain;

import java.math.BigDecimal;
import java.util.List;
import java.util.UUID;

public class JournalEntry {
    private final UUID id;
    private final String idempotencyKey;
    private final List<Posting> postings;

    public JournalEntry(UUID id, String idempotencyKey, List<Posting> postings) {
        this.id = id;
        this.idempotencyKey = idempotencyKey;
        this.postings = List.copyOf(postings);
        validateDoubleEntryBalance();
    }

    private void validateDoubleEntryBalance() {
        long totalSignedCents = postings.stream()
            .mapToLong(Posting::getSignedAmountCents)
            .sum();
        if (totalSignedCents != 0) {
            throw new IllegalStateException("Violação da Contabilidade Dupla: a
soma das lançamentos deve ser ZERO.");
        }
    }
}
```

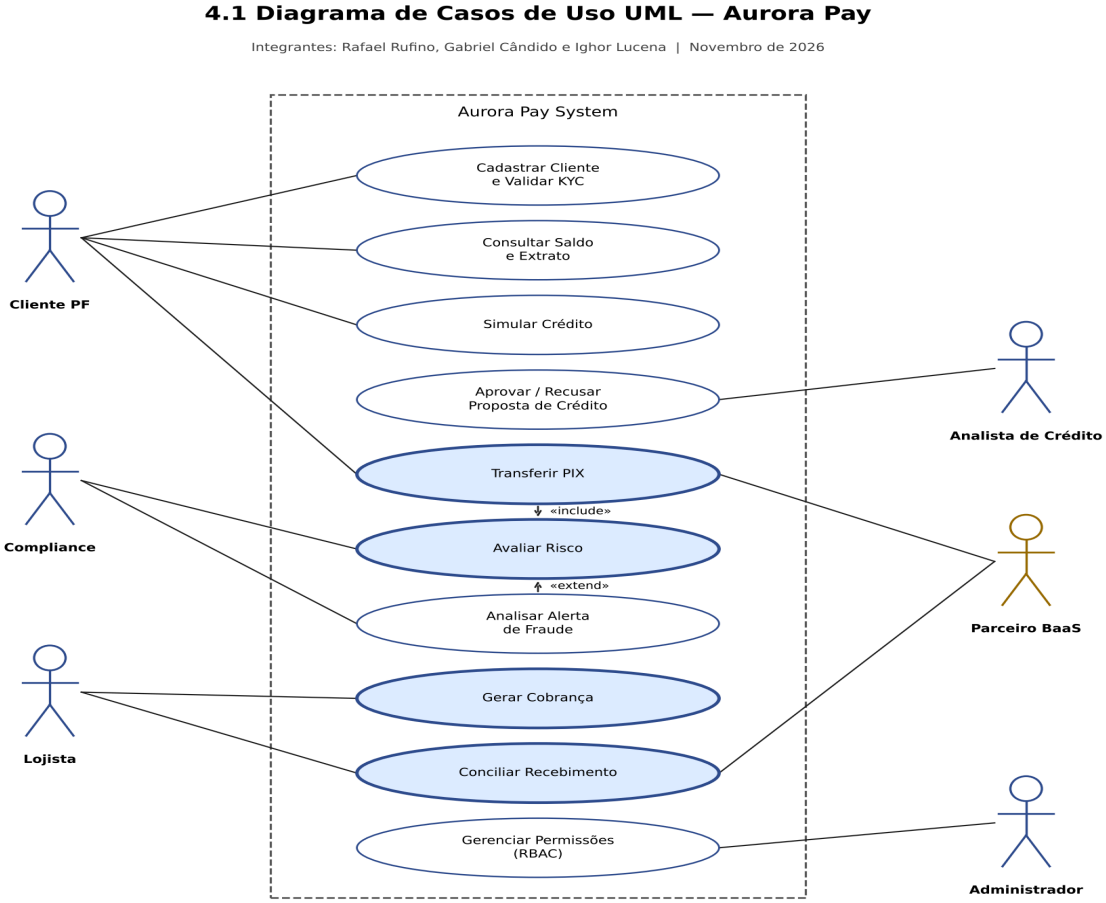
---

# BLOCO 4 — DIAGRAMAS VISUAIS UML (MODELADOS NO DRAW.IO)

*Todos os diagramas apresentados contêm Título, Nomes dos Integrantes e Data de Novembro de 2026, conforme norma do trabalho.*

## 4.1 Diagrama de Casos de Uso UML

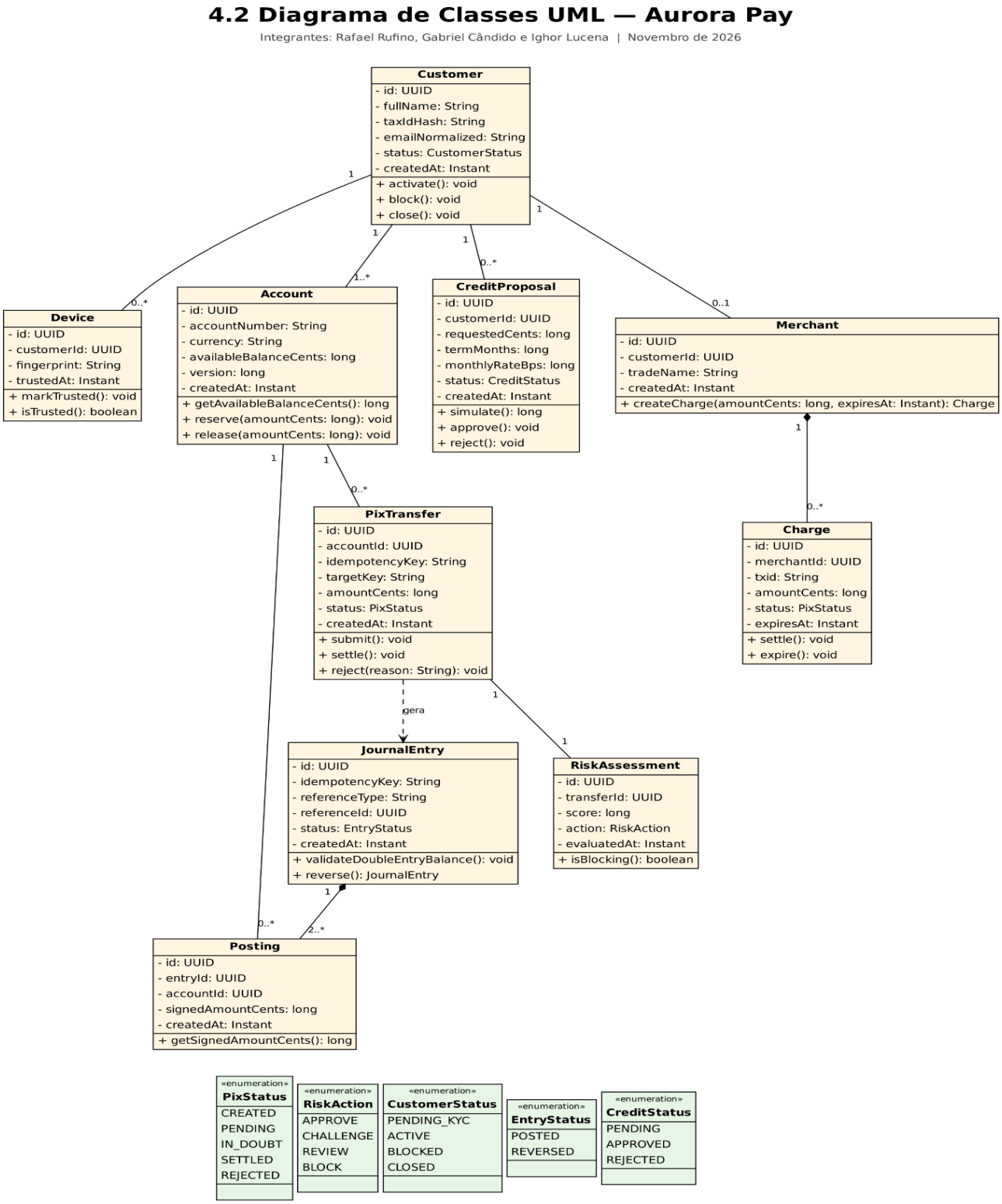
Apresenta os atores (`Cliente PF`, `Lojista`, `Analista de Crédito`, `Compliance`, `Administrador` e `Parceiro BaaS`) e as relações com os casos de uso principais (`Transferir PIX`, `Avaliar Risco`, `Gerar Cobrança`, `Conciliar Recebimento`).




## 4.2 Diagrama de Classes UML

Contém todas as entidades do núcleo com atributos tipados (`UUID`, `String`, `long`, `Instant`, `enum`) e métodos com visibilidade explícita (`+`, `-`): Entidades: `Customer`, `Device`, `Account`, `JournalEntry`, `Posting`, `PixTransfer`, `RiskAssessment`, `CreditProposal`,

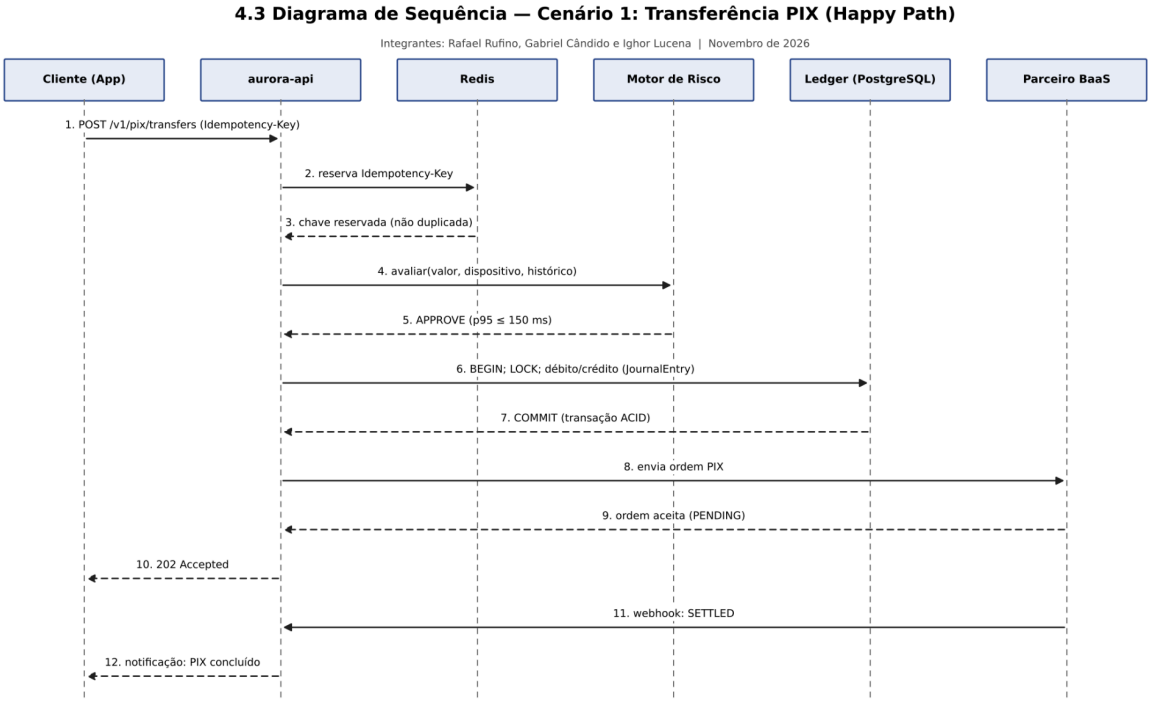
`merchant`, `Charge`.



## 4.3 Diagramas de Sequência UML (3 Cenários Obrigatórios Exigidos)

### Cenário 1: Transferência PIX (Happy Path)

Detalha o fluxo completo do cliente informando o PIX, verificação no Redis, avaliação do engine de risco (retornando `APPROVE`), reserva no Ledger em transação ACID e confirmação com o BaaS parceiro.

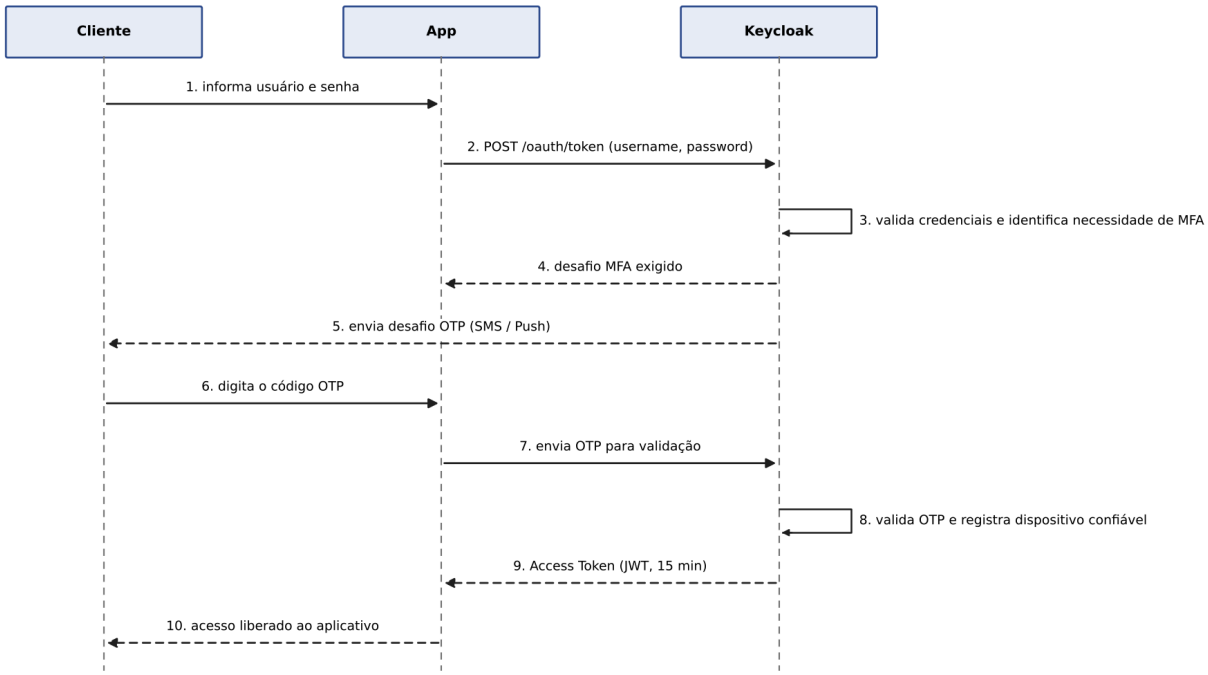



### Cenário 2: Autenticação Segura OIDC / MFA (Fluxo de Login)

1. `App` envia credenciais `(username, password)` para `/oauth/token` no `Keycloak`.
2. `Keycloak` valida e identifica necessidade de MFA para o dispositivo.
3. `Keycloak` envia desafio OTP (SMS/Push); `Cliente` digita o código no `App`.
4. `Keycloak` valida o OTP, registra o dispositivo confiavel e retorna o `Access Token (JWT, 15 min)`.

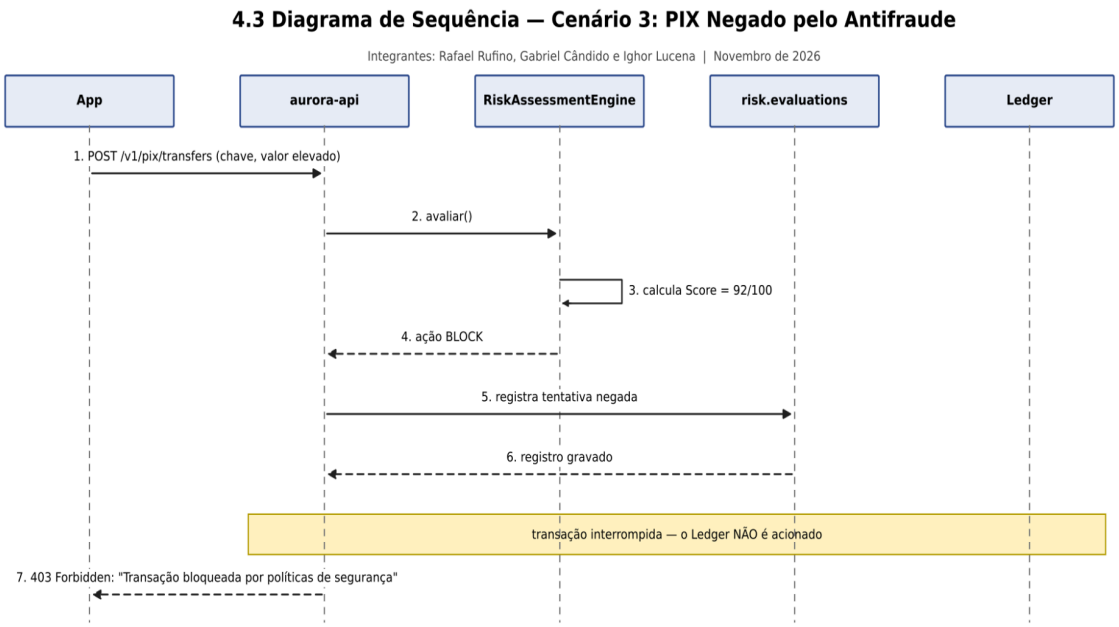
**4.3 Diagrama de Sequência — Cenário 2: Autenticação Segura OIDC / MFA**

Integrantes: Rafael Rufino, Gabriel Cândido e Ighor Lucena | Novembro de 2026




## Cenário 3: PIX Negado por Rejeição do Antifraude (Cenário de Erro Crítico)

1. `App` solicita `POST /v1/pix/transfers` com chave e valor elevado.
2. `aurora-api` invoca o `RiskAssessmentEngine`.
3. `RiskAssessmentEngine` calcula o score de risco (Score = 92/100) e retorna a ação `BLOCK`.
4. `aurora-api` registra a tentativa negada no schema `risk.evaluations`.
5. `aurora-api` interrompe a transação (sem tocar no Ledger) e retorna HTTP `403 Forbidden` com a mensagem `"Transação bloqueada por políticas de segurança"`.

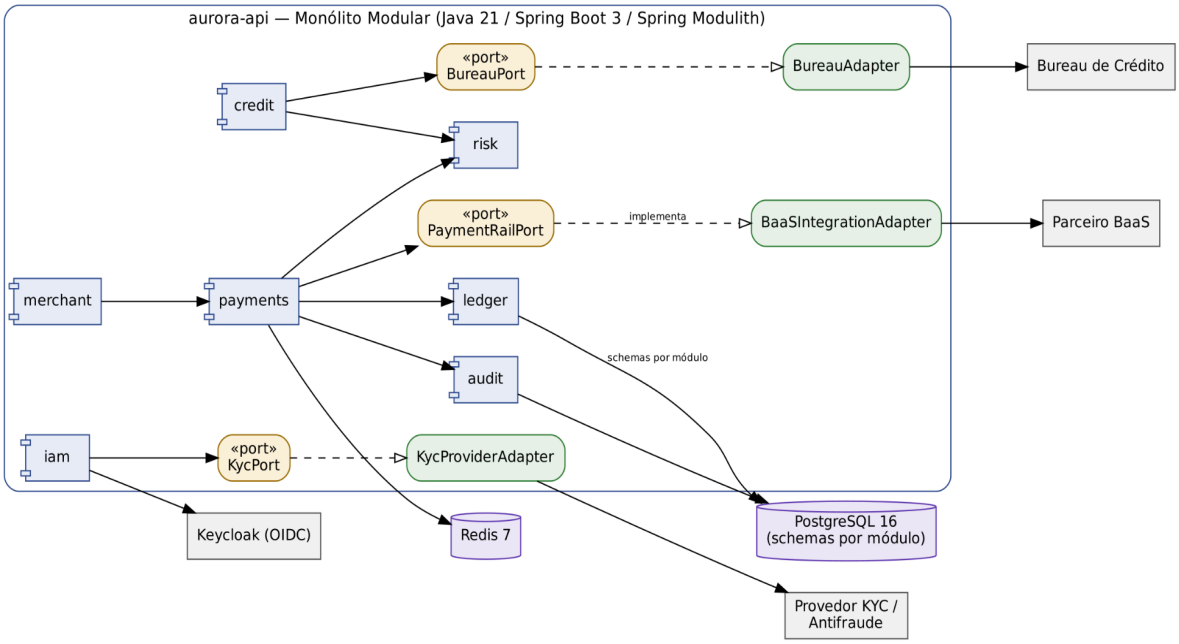



## 4.4 Diagrama de Componentes UML

Estrutura modular interna do Monólito em Java/Spring e portas adaptadoras de integração (`PaymentRailPort`, `KycPort`, `BureauPort`).

## 4.4 Diagrama de Componentes UML — Aurora Pay

Integrantes: Rafael Rufino, Gabriel Cândido e Ighor Lucena | Novembro de 2026



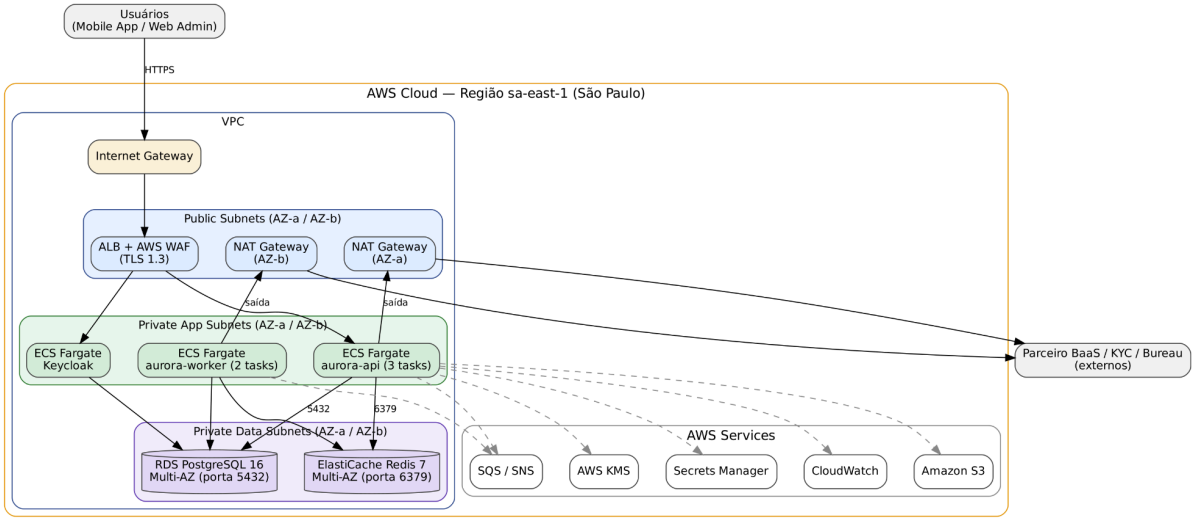

## 4.5 Diagrama de Implantação (Deployment) AWS sa-east-1

Detalha a TOPOLOGIA Cloud na região São Paulo (sa-east-1):

- **Public Subnets (AZ-a / AZ-b):** Internet Gateway, ALB + WAF, NAT Gateways.
- **Private App Subnets (AZ-a / AZ-b):** Tasks ECS Fargate (`aurora-api`, `aurora-worker`, `Keycloak`).
- **Private Data Subnets (AZ-a / AZ-b):** RDS PostgreSQL 16 Multi-AZ (Porta 5432), ElastiCache Redis Multi-AZ (Porta 6379).
- **AWS Services Conectados:** Amazon S3, SQS/SNS, AWS KMS, Secrets Manager e CloudWatch.

**4.5 Diagrama de Implantação (Deployment) — AWS sa-east-1**

Integrantes: Rafael Rufino, Gabriel Cândido e Ighor Lucena | Novembro de 2026

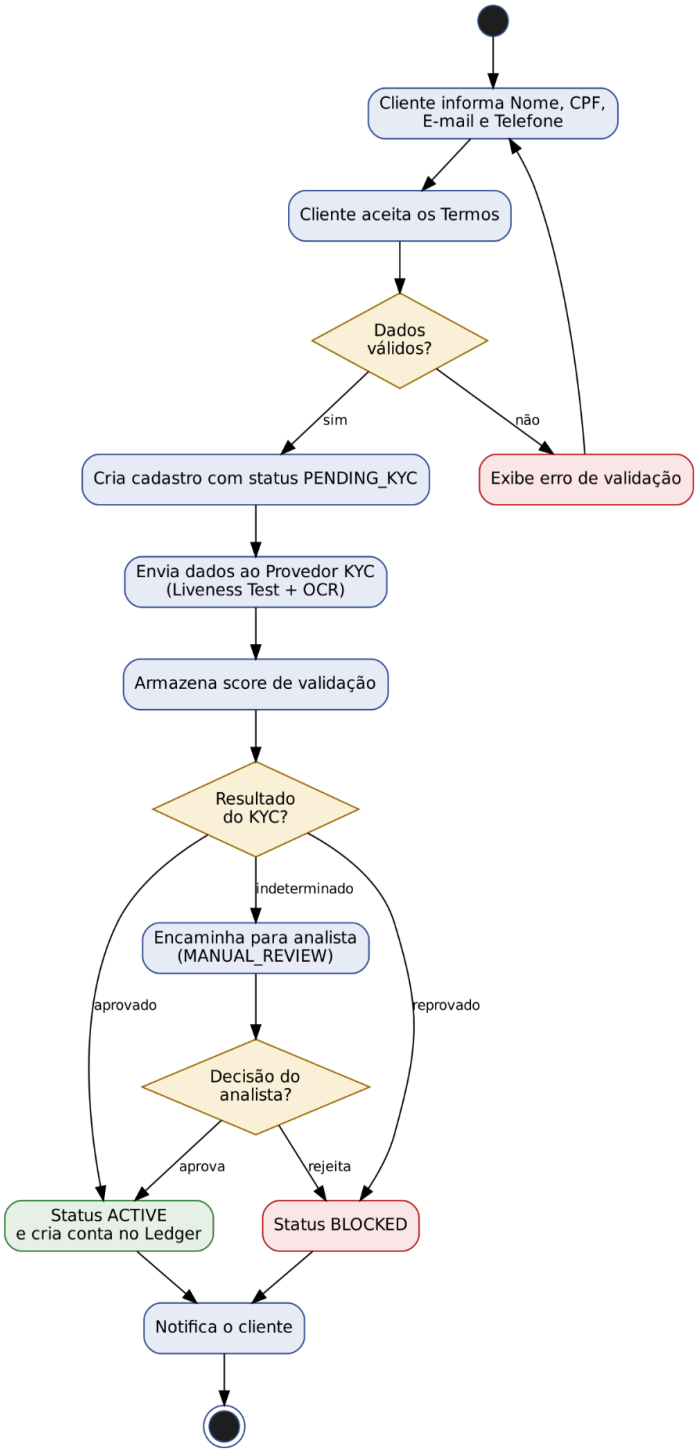



## 4.6 Diagramas de Atividades UML (2 Processos Obrigatórios Exigidos)

1. **Processo de Negócio Principal:** Fluxo completo de Onboarding e KYC do cliente.

# 4.6 Diagrama de Atividades — Onboarding e KYC do Cliente

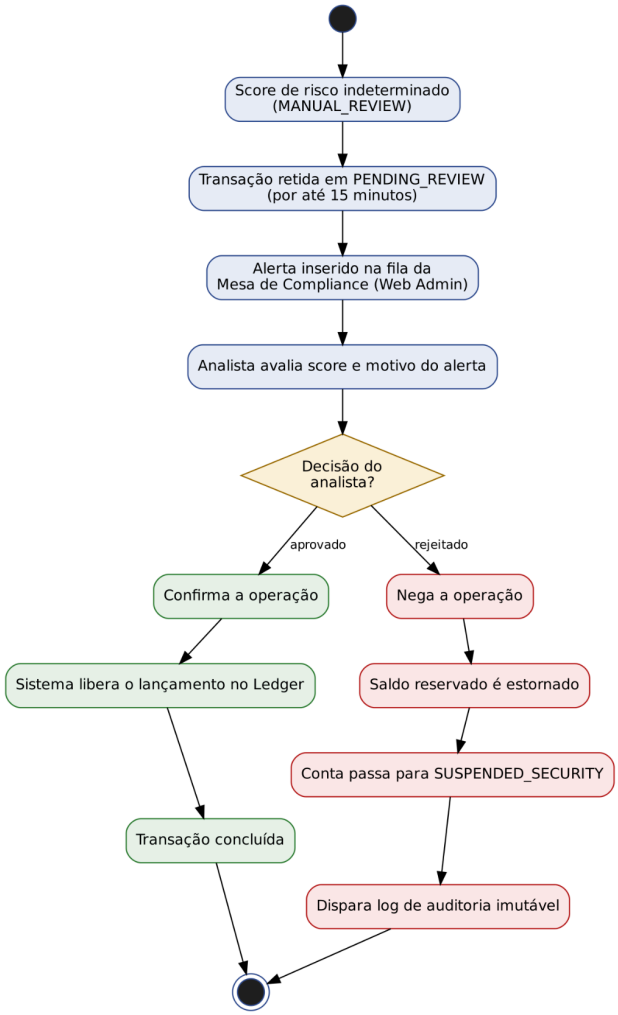
Integrantes: Rafael Rufino, Gabriel Cândido e Ighor Lucena | Novembro de 2026



2. **Processo Administrativo Relevante (Análise Manual de Suspeita de Fraude na Mesa de Compliance):**
    - *Início:* Evento de score de risco indeterminado (`MANUAL_REVIEW`).
    - *Ação:* Transação fica retida em estado `PENDING_REVIEW` por até 15 minutos.
    - *Análise:* Alerta é inserido na fila da Mesa de Compliance do Web Admin.
    - *Decisão do Analista:*
        - Se *Aprovado:* O analista confirma a operação $\rightarrow$ O sistema libera o lançamento no Ledger $\rightarrow$ Transação é concluída.
        - Se *Rejeitado:* O analista nega a operação $\rightarrow$ O saldo reservado é estornado $\rightarrow$ A conta do cliente passa para status `SUSPENDED_SECURITY` $\rightarrow$ Dispara log de auditoria imutável.

## 4.6 Diagrama de Atividades — Análise Manual de Suspeita de Fraude

Integrantes: Rafael Rufino, Gabriel Cândido e Ighor Lucena | Novembro de 2026

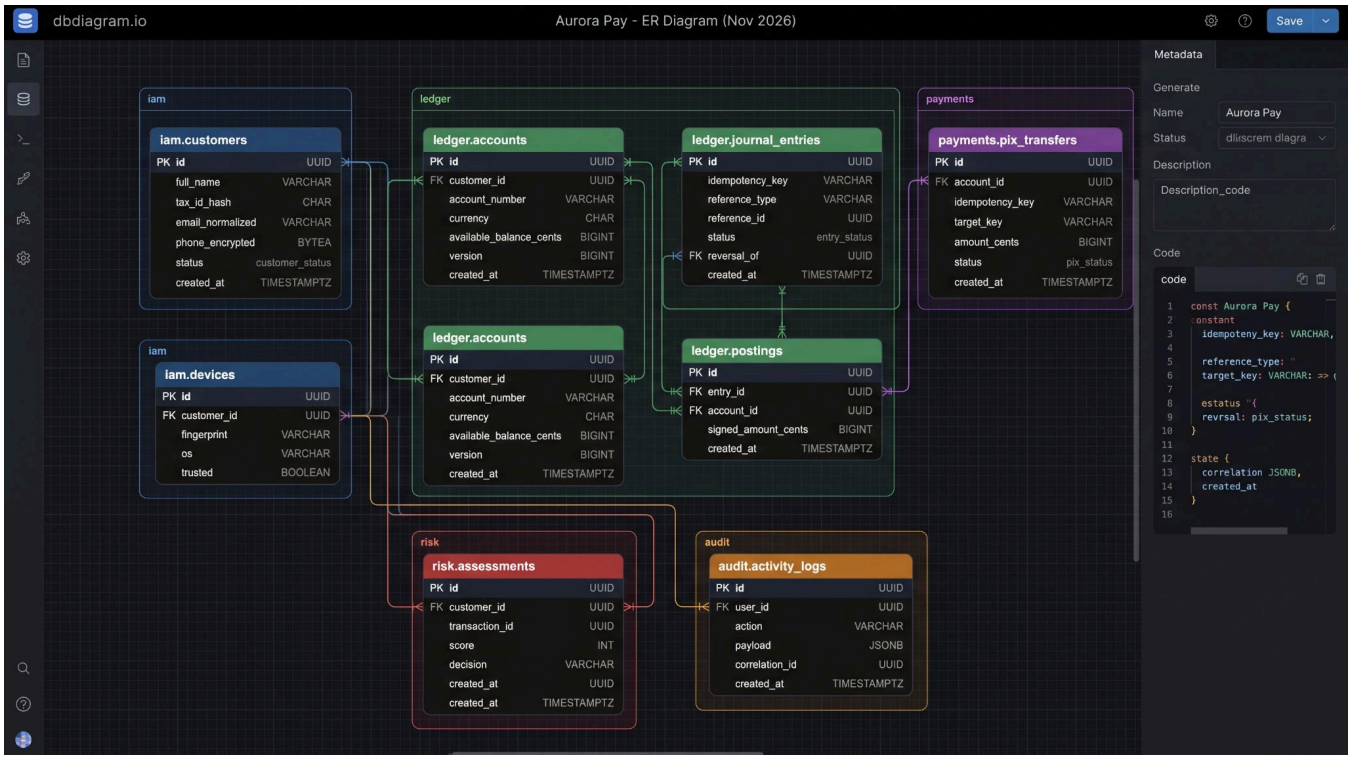


# BLOCO 5 — BANCO DE DADOS E MODELO ER (VALIDADES NO PGADMIN)

## 5.1 Diagrama Entidade-Relacionamento (ER Conceitual) e Modelo Lógico

O modelo de dados é estruturado sob normalização rigorosa até a 3ª Forma Normal (3FN), segregado por schemas funcionais no PostgreSQL 16:

- `iam`: Clientes, dispositivos, permissões e consentimentos LGPD.
- `ledger`: Contas financeiras, lançamentos contábeis (`journal_entries`) e débitos/créditos (`postings`).
- `payments`: Chaves PIX, transações PIX, boletos e webhooks.
- `risk`: Avaliações de risco, scores e alertas de compliance.
- `platform`: Configurações de sistema e tabelas de outbox.
- `audit`: Trilhas de auditoria append-only.



## 5.2 Dicionário de Dados com Marcação Explícita de Tratamento LGPD (PII)

| Schema | Tabela | Campo | Tipo de Dado | Restrições (Constraints) | Tratamento LGPD / PII | Descrição de Negócio |
|---|---|---|---|---|---|---|
| iam | customers | id | UUID | PRIMARY KEY | Não PII | Identificador único do cliente. |
| iam | customers | full_name | VARCHAR(150) | NOT NULL | **PII (Dado Pessoal)** | Nome completo do cliente titular. |
| iam | customers | tax_id_hash | CHAR(64) | NOT NULL, UNIQUE | **PII Pseudonimizado** | Hash HMAC-SHA256 do CPF do cliente. |
| iam | customers | email_normalized | VARCHAR(254) | NOT NULL, UNIQUE | **PII (Dado Pessoal)** | E-mail de contato do usuário. |
| iam | customers | phone_encrypted | BYTEA | NOT NULL | **PII Criptografado** | Telefone em formato criptografado (AES-256). |
| ledger | accounts | id | UUID | PRIMARY KEY | Não PII | Identificador único da conta. |
| ledger | accounts | customer_id | UUID | FK -> customers.id | Não PII | Vínculo com a tabela de clientes. |

| Schema | Tabela | Campo | Tipo de Dado | Restrições (Constraints) | Tratamento LGPD / PII | Descrição de Negócio |
|---|---|---|---|---|---|---|
| ledger | accounts | available_balance_cents | BIGINT | CHECK (>= 0) | Não PII | Saldo disponível em centavos de BRL. |
| ledger | postings | signed_amount_cents | BIGINT | CHECK (<> 0) | Não PII | Valor assinado do lançamento contábil. |

## 5.3 Código DDL Fisico Executável no PostgreSQL 16 (Testado via pgAdmin)

```sql
-- DDL EXECUTÁVEL COMPLETO DA FINTECH AURORA PAY (POSTGRESQL 16)
CREATE EXTENSION IF NOT EXISTS pgcrypto;

CREATE SCHEMA IF NOT EXISTS iam;
CREATE SCHEMA IF NOT EXISTS ledger;
CREATE SCHEMA IF NOT EXISTS payments;
CREATE SCHEMA IF NOT EXISTS risk;
CREATE SCHEMA IF NOT EXISTS platform;
CREATE SCHEMA IF NOT EXISTS audit;

-- ENUMs
CREATE TYPE iam.customer_status AS ENUM ('PENDING_KYC', 'ACTIVE', 'BLOCKED',
'CLOSED');
CREATE TYPE ledger.entry_status AS ENUM ('POSTED', 'REVERSED');
CREATE TYPE payments.pix_status AS ENUM ('CREATED', 'PENDING', 'IN_DOUBT',
'SETTLED', 'REJECTED');

-- TABELA CUSTOMERS (IAM)
CREATE TABLE iam.customers (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    full_name VARCHAR(150) NOT NULL,
    tax_id_hash CHAR(64) NOT NULL UNIQUE,
    email_normalized VARCHAR(254) NOT NULL UNIQUE,
    phone_encrypted BYTEA NOT NULL,
```

```
    keycloak_user_id VARCHAR(100) UNIQUE,
    status iam.customer_status NOT NULL DEFAULT 'PENDING_KYC',
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- TABELA ACCOUNTS (LEDGER)
CREATE TABLE ledger.accounts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    customer_id UUID NOT NULL REFERENCES iam.customers(id) ON DELETE RESTRICT,
    account_number VARCHAR(24) NOT NULL UNIQUE,
    currency CHAR(3) NOT NULL DEFAULT 'BRL',
    available_balance_cents BIGINT NOT NULL DEFAULT 0 CHECK
(available_balance_cents >= 0),
    version BIGINT NOT NULL DEFAULT 1,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- TABELA JOURNAL_ENTRIES (LEDGER)
CREATE TABLE ledger.journal_entries (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    idempotency_key VARCHAR(100) NOT NULL UNIQUE,
    reference_type VARCHAR(40) NOT NULL,
    reference_id UUID NOT NULL,
    status ledger.entry_status NOT NULL DEFAULT 'POSTED',
    reversal_of UUID REFERENCES ledger.journal_entries(id),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- TABELA POSTINGS (LEDGER - DUPLA ENTRADA)
CREATE TABLE ledger.postings (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    entry_id UUID NOT NULL REFERENCES ledger.journal_entries(id) ON DELETE
RESTRICT,
    account_id UUID NOT NULL REFERENCES ledger.accounts(id) ON DELETE RESTRICT,
    signed_amount_cents BIGINT NOT NULL CHECK (signed_amount_cents <> 0),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ÍNDICES DE ALTA PERFORMANCE
CREATE INDEX ix_postings_account_date ON ledger.postings(account_id, created_at
DESC);
CREATE INDEX ix_journal_entries_idempotency ON
ledger.journal_entries(idempotency_key);
```

---

# BLOCO 7 — PLANEJAMENTO, WBS E ESTIMATIVAS DE CUSTOS REAIS

## 7.1 Estrutura Analítica do Projeto (WBS - 3 Níveis)

1. **Aurora Pay System**
    - **1.1 Concepção e Governança**
        - 1.1.1 Documento de Visão e Requisitos
        - 1.1.2 Parecer Jurídico LGPD e BaaS
    - **1.2 Arquitetura e Modelagem**
        - 1.2.1 Especificação de ADRs e Modelo C4
        - 1.2.2 Modelagem de Banco de Dados ER/DDL
    - **1.3 Desenvolvimento Core & Backend**
        - 1.3.1 Módulo IAM & Integração Keycloak
        - 1.3.2 Motor Contábil Double-Entry Ledger
        - 1.3.3 Módulo PIX e Conciliação Lojista
    - **1.4 Frontends Mobile e Web**
        - 1.4.1 Aplicativo React Native (Cliente PF/PJ)
        - 1.4.2 Painel Web Admin em React/Vite (Compliance)
    - **1.5 DevOps, QA e Handoff**
        - 1.5.1 Pipeline CI/CD & AWS Infrastructure
        - 1.5.2 Testes de Carga k6 & Documentação OpenAPI

## 7.2 Pesquisa Salarial Datada e Custo de Equipe (`glassdoor.com.br` - Novembro 2026)

| **Cargo / Papel** | **Senioridade** | **Qtd** | **Salário Bruto Mensal** | **Encargos + Benefícios (+70%)** | **Custo Total Mensal** |
|---|---|---|---|---|---|
| **Tech Lead / Arquiteto** | Sênior | 1 | R$ 15.000,00 | R$ 10.500,00 | R$ 25.500,00 |
| **Dev Backend (Java)** | Pleno/Sr | 1 | R$ 10.000,00 | R$ 7.000,00 | R$ 17.000,00 |
| **Dev Mobile / Web** | Pleno | 1 | R$ 9.000,00 | R$ 6.300,00 | R$ 15.300,00 |

| Cargo / Papel | Senioridade | Qtd | Salário Bruto Mensal | Encargos + Benefícios (+70%) | Custo Total Mensal |
|---|---|---|---|---|---|
| **Engenheiro DevOps** | Pleno/Sr | 1 | R$ 11.000,00 | R$ 7.700,00 | R$ 18.700,00 |
| **Engenheiro de QA** | Pleno | 1 | R$ 7.000,00 | R$ 4.900,00 | R$ 11.900,00 |
| **Product Owner (PO)** | Pleno/Sr | 1 | R$ 9.000,00 | R$ 6.300,00 | R$ 15.300,00 |
| **TOTAL MENSAL SQUAD** | - | **6** | **R$ 61.000,00** | **R$ 42.700,00** | **R$ 103.700,00** |

- **Custo Total de Pessoal do Projeto (3 Meses / 6 Sprints): R$ 311.100,00**.

## 7.3 Export Detalhado da AWS Pricing Calculator (`calculator.aws` - Região São Paulo `sa-east-1`)

| Componente AWS | Especificação Técnica Dimensionada | Custo Mensal (USD) |
|---|---|---|
| **AWS ECS Fargate** | 4 vCPU, 8 GB RAM (3 Tasks API + 2 Tasks Worker sustentadas) | US$ 215,40 |
| **AWS RDS PostgreSQL 16** | db.r6g.xlarge Multi-AZ, 250 GB GP3 Storage | US$ 542,10 |
| **AWS ElastiCache Redis** | cache.m6g.large Multi-AZ | US$ 148,00 |
| **ALB + AWS WAF** | 1 Application Load Balancer + Regras OWASP ativas | US$ 85,20 |
| **NAT Gateways** | 2 NAT Gateways (1 por AZ) + Processamento de Dados | US$ 112,00 |
| **Amazon S3 & KMS** | Storage Documentos KYC + Operações KMS Criptografia | US$ 28,50 |

| Componente AWS | Especificação Técnica Dimensionada | Custo Mensal (USD) |
|---|---|---|
| **CloudWatch & Logs** | Ingestão de Logs e Tracing OpenTelemetry | US$ 45,00 |
| **SNS, SQS & Transfer** | Filas SQS FIFO, Mensageria SNS e Saída de Dados | US$ 34,80 |
| **SUBTOTAL PRODUÇÃO** | **Ambiente de Alta Disponibilidade em Produção** | **US$ 1.211,00** |
| **STAGING ENVIRONMENT** | ECS reduzido (1 task), RDS Single-AZ | US$ 310,00 |
| **DEV ENVIRONMENT** | ECS mínimo com desligamento noturno configurado | US$ 110,00 |
| **CUSTO TOTAL AWS MENSAL** | **Soma dos 3 Ambientes Operacionais (Dev + Staging + Prod)** | **US$ 1.631,00** |

---

BLOCO 8 — DOCUMENTAÇÃO PARA O DESENVOLVEDOR (HANDOFF DIA 1)

## 8.1 OpenAPI 3.0 — Contrato de API PIX (Swagger Format)

```yaml
openapi: 3.0.3
info:
  title: Aurora Pay Core API
  version: 1.0.0
paths:
  /pix/transfers:
    post:
      summary: Inicia uma transferência PIX com garantia de idempotência
      parameters:
        - in: header
          name: Idempotency-Key
          required: true
          schema:
            type: string
            format: uuid
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [accountId, targetKey, amountCents]
              properties:
                accountId:
                  type: string
                  format: uuid
                targetKey:
                  type: string
                amountCents:
                  type: integer
                  minimum: 1
      responses:
        '202':
          description: Transferência aceita e enviada para processamento.
        '403':
          description: Operação bloqueada pelo Motor de Risco / Antifraude.
        '409':
          description: Conflito de idempotência (requisição duplicada).
```

# 8.2 README Completo

## Aurora Pay - Core API

### Descrição

Motor principal de pagamentos da Aurora Pay, responsável pelo processamento de transferências PIX, validação de saldo e integração com o motor antifraude.

**Pré-requisitos**

- Node.js (v18+)
- Docker e Docker Compose (para o banco de dados local)
- Git

**Como rodar o projeto localmente (Localhost)**

1. Clone o repositório:
   git clone https://github.com/aurorapay/core-api.git
2. Acesse a pasta do projeto:
   cd core-api
3. Configure as variáveis de ambiente:
   cp .env.example .env
4. Suba a infraestrutura local (Banco de Dados PostgreSQL):
   docker-compose up -d
5. Instale as dependências:
   npm install
6. Rode as migrations do banco de dados:
   npm run typeorm migration:run
7. Inicie o servidor em modo de desenvolvimento:
   npm run dev

A API estará disponível em http://localhost:3000. Acesse http://localhost:3000/api-docs para visualizar o contrato OpenAPI (Swagger).

# 8.3 Variáveis de Ambiente (.env)

```
# Configurações da Aplicação
PORT=3000
NODE_ENV=development

# Banco de Dados (PostgreSQL)
DB_HOST=localhost
DB_PORT=5432
DB_USER=aurora_dev
DB_PASS=aurora_dev_password
DB_NAME=aurora_core_db

# Integrações e Segurança
JWT_SECRET=sua_chave_secreta_jwt_aqui
ANTI_FRAUD_SERVICE_URL=http://localhost:3001/verify
BACEN_PIX_API_KEY=mock_key_para_desenvolvimento_local
```

# 8.4 Guia de Padrões de Código

Arquitetura: Utilizamos Clean Architecture (Arquitetura Limpa). O código deve ser estritamente dividido nas camadas de Domain (Entidades), UseCases (Regras de negócio), Adapters (Controladores/Interfaces) e Infrastructure (Banco de Dados/APIs externas).

Convenção de Nomes:
- Arquivos e Pastas: kebab-case (ex: pix-transfer.controller.ts)
- Classes e Interfaces: PascalCase (ex: PixTransferService)
- Variáveis e Métodos: camelCase (ex: processTransfer())
- Constantes: UPPER_SNAKE_CASE (ex: MAX_TRANSFER_LIMIT)

Linting e Formatação: O projeto utiliza ESLint e Prettier. O commit será bloqueado pelo Husky caso haja erros de lint (comando npm run lint).

Tratamento de Erros: Nunca retorne erros genéricos. Utilize a classe padrão AppError passando o status HTTP e a mensagem mapeada para o front-end.

# 8.5 Git Workflow Definido

Estratégia: Utilizaremos o GitFlow.

Branches Principais:
- main: Código em produção (estável).
- develop: Código em ambiente de staging (integração).

Branches de Trabalho:
- feature/nome-da-feature (ex: feature/pix-idempotency)
- bugfix/nome-do-bug (ex: bugfix/fix-pix-timeout)

Padrão de Commits (Conventional Commits):
- feat: adiciona endpoint de transferência PIX
- fix: corrige validação de saldo insuficiente
- docs: atualiza documentação do swagger

Pull Requests (PRs): Para fazer merge em develop, o PR exige no mínimo 1 aprovação (Code Review) de outro desenvolvedor e que a esteira de CI (testes automatizados) passe com sucesso.

# 8.6 Sprint 1 Detalhada com Critérios de Aceite

Objetivo da Sprint 1: Implementar o motor básico de processamento de PIX com garantia de idempotência e validação de risco.

## User Story 1 (US01): Inicialização de Transferência PIX

História: Como cliente (persona), quero realizar uma transferência PIX informando a chave de destino e o valor, para pagar terceiros rapidamente.

Critérios de Aceite (BDD):

Cenário 1: Transferência válida.
- Given que o cliente possui saldo de R$ 100,00.
- When ele solicitar um PIX de R$ 50,00 para uma chave válida.
- Then o sistema deve retornar status HTTP 202 (Aceito) e deduzir o saldo.

Cenário 2: Saldo insuficiente.
- Given que o cliente possui saldo de R$ 20,00.
- When ele solicitar um PIX de R$ 50,00.
- Then o sistema deve bloquear a operação e retornar status HTTP 422 com a mensagem "Saldo insuficiente".

## User Story 2 (US02): Proteção contra Duplicidade (Idempotência)

História: Como desenvolvedor back-end, quero garantir que a mesma requisição PIX não seja processada duas vezes em caso de falha de rede (retry), para evitar prejuízos financeiros.

Critérios de Aceite (BDD):

Cenário 1: Primeira tentativa.
- Given um payload de transferência com o Idempotency-Key: 123-abc.
- When a API receber a requisição pela primeira vez.
- Then o sistema deve processar o PIX e salvar a chave no banco de dados.

Cenário 2: Tentativa duplicada.
- Given que a chave 123-abc já foi processada com sucesso há 5 segundos.
- When a API receber um novo payload com a mesma Idempotency-Key.
- Then o sistema não deve refazer a transação financeira e deve retornar o status HTTP 409 (Conflito), conforme documentado no OpenAPI.

---

# BLOCO 9 — QUALIDADE, PROCESSO E CRITÉRIOS DE ACEITE

## 9.1 Definition of Ready (DoR) e Definition of Done (DoD)

- **Definition of Ready (DoR):**
    1. História descrita com persona, ação e valor de negócio claro.
    2. Critérios de aceite especificados no formato BDD (`Given/When/Then`).
    3. Contrato de API / Swagger definido para a funcionalidade.
    4. Estimativa realizada em Story Points e aprovada por todo a squad.
- **Definition of Done (DoD):**
    1. Código implementado sem alertas de linter e com padrões da arquitetura.
    2. Cobertura de testes unitários e de integração >= 80% no módulo alterado.
    3. Pull Request revisado e aprovado por no mínimo 1 desenvolvedor sênior.
    4. Pipeline CI/CD executada com sucesso (Build, Testes, SAST e Sonar).
    5. Funcionalidade validada no ambiente de Staging.